HybridFuzz Demo

Chen Chen

Date: 08/20/2022

Introduction

The purpose of this Demo is to show the basic functions of HybridFuzz, including how to identify uncovered points from a coverage report of TheHuzz, how to identify the conditions of covering each uncovered point, and how to convert the conditions into cover property as inputs of Jaspergold. The Demo also includes the algorithms implemented to determine when to switch to formal and how to select uncovered points for formal tools. All examples in this demo are based on the CVA6 processor.

File Hierarchy

cov_html: a human-readable coverage report; used to understand the coverage information from TheHuzz; not related to any functions of HybridFuzz.

CVA6CoreBlackbox.sv: contains the source code of CVA6 processor; used to understand the coverage information; not related to any functions of HybridFuzz.

cov_xml: a coverage report in xml format; generated by VCS after simulating a test case; HybridFuzz mainly parses it to obtain useful information.

nop.*: three different formats of test cases generated by TheHuzz. Used to demonstrate the format of test cases and the method to restrict the initial state of a DUT in Jaspergold. Not related to any functions of HybridFuzz.

> *.c: a C program like stimulus generated by TheHuzz.
>
> *.hex: the format that mutation will happen.
>
> *.mem: the format taken by the simulation environment.
>
> Note: the coverage report is generated by only simulating the **nop** test case, with only nop instructions for testing. The original purpose to provide such test case is to reduce the utilization of Jaspergold. If points can be covered during the initialization of simulation, there is no need for formal. In real case, TheHuzz will merge coverage reports after simulating all test cases.

***.py**: functions of HybridFuzz

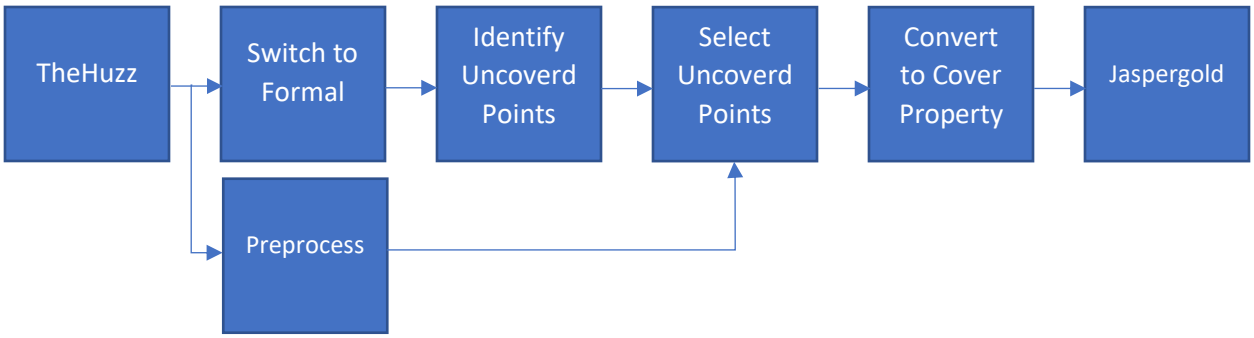
> HybridFuzz.py: main function; contains logic of all basic functions and strategies
>
> AutoParse.py, parse_pkg.py : contains logic to parse coverage report
>
> Config.py: configurations

***.csv, *.xlsl**: rank tables of modules in CVA6 for point selection strategies: BotTop, TopBot, ModDep.

uncovd_point.json: contains information parsed from the coverage report, including hierarchical path of each module, number of uncovered points, and the conditions of covering each point.

Flow-chart:

The flow chart shows the basic flow of HybridFuzz.



Preprocess: analyzes design information from the coverage report of TheHuzz. Generate rank tables of modules for point selection strategies: ModDep, TopBot, BotTop.

> ModDep: Let Jaspergold to analyze the fanouts of each module's outputs cross the entire design (cone of influence). Rank modules based on the highest fanouts of each module.
>
> TopBot, BotTop: analyze the module hierarchy from the coverage report. Rank modules based on their distances to the top module.

Switch to Formal: compare the rate of fuzzing and formal. If the rate of formal is higher, switch to formal

Identify Uncovered Points: Identify uncovered branch points distributed in each module.

Select Uncovered Points:

> Randsel: randomly select uncovered points.
>
> Maxuncovd: randomly select points from a module with maximal uncovered points.
>
> ModDep: randomly select points from a module with the highest fanout.
>
> TopBot, BotTop: randomly select points from a module that is closest or farthest to the top module, respectively.

Convert to Cover Property: convert the conditions of selected point into a cover SVA property.

Demo

Command to run the script: $python3 HybridFuzz.py

Outputs:

```
:: python3 HybridFuzz.py
======Preprocess design information======
number of instances of the entire SoC environment:  1144
number of instaces belong to the DUT:  321
======Compare rate and determine if it is the time to switch to formal======
        Rate of fuzzing:  0.01
        Rate of formal:  0.48192771084337344
        Switch to formal:  True
======Identify uncovered points in a DUT======
Total points of branch coverage:  9533
Total covered points of branch coverage:  4798
======point selection strategy======
point selection strategy:  randsel
cover property 960_39 {sum_shifted[((3 * PRECISION_BITS) + 4)]}
point selection strategy:  maxuncovd
cover property 1139_548 {defs_div_sqrt_mvp::Iteration_unit_num_S == 2'b11 && Crtl_cnt_S == 6'b000111 && !(Sqrt_quotinent
_S[1])}
point selection strategy:  bottop
cover property 525_0 {Div_enable_SI}
```

Output explanation:

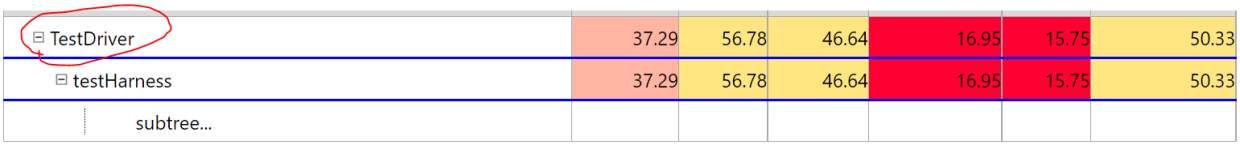
1. **Preprocess design information:** This step will generate rank tables of modules. The coverage reports contain modules of the entire simulation environment. It will only analyze modules belonging to the DUT.
2. **Compare rate …:** as it shown
3. **Identify uncovered points in a DUT:** The function will identify the total branch points in each module and sum them for the total branch points of the DUT. It will also identify the uncovered points in each module, sum them to count the total covered points of the DUT.
4. **Point selection strategy:** It will show the uncovered point select by each strategy and the cover SVA property that will be loaded to Jaspergold. The property contains the conditions of covering that point.

Output justification:

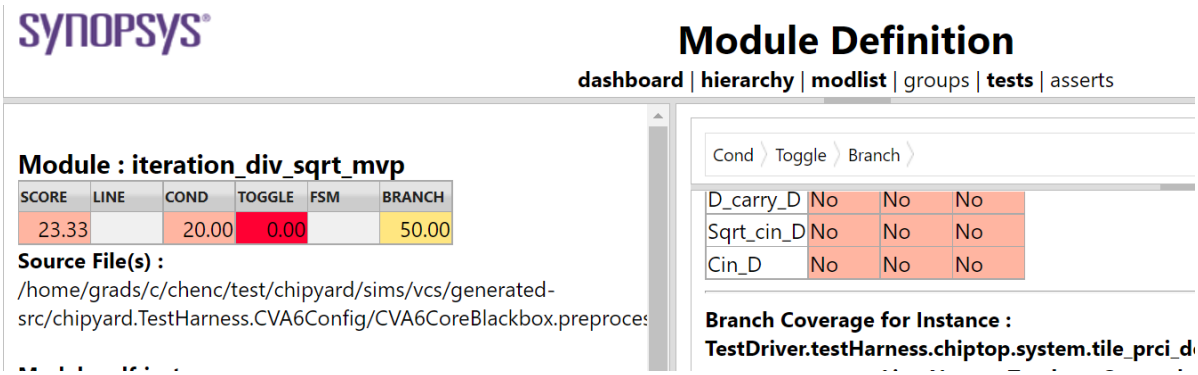
1. Example: point selection strategy: bottop; cover property 525_0 {DIV_enable_SI}; the property identified one uncovered point in the module and tries to set the signal *DIV_enable_SI* to 1 to cover the point.
2. Format of cover property: cover property *instanceID_pointID* {*conditions of the point*}
3. Find instanceID:
    a. open *uncovd_point.json*
    b. search "inst_id": "-525"
    c. The information of this module is shown:

```
"TestDriver.testHarness.chiptop.system.tile_prci_domain.
  "inst_name": "genblk4[2].iteration_div_sqrt",
  "inst_id": "-525",
  "inst_pid": "-1139",
  "module_name": "iteration_div_sqrt_mvp",
  "mod_id": "-197",
  "uncovd_points": {
    "0": "Div_enable_SI"
  }
},
```

4. Find the hierarchical path of the module as its key:
   TestDriver.testHarness.chiptop.system.tile_prci_domain.tile_reset_domain.cva6_tile.core.i_ariane.ex_stage_i.fpu_gen.fpu_i.fpu_gen.i_fpnew_bulk.gen_operation_groups[1].i_opgroup_block.gen_merged_slice.i_multifmt_slice.gen_num_lanes[0].active_lane.lane_instance.i_fpnew_divsqrt_multi.i_divsqrt_lei.nrbd_nrsc_U0.control_U0.genblk4[2].iteration_div_sqrt
5. Locate this module in the coverage report **cov_html**:
   a. Open *cov_html/hierarchy.html*
   b. Go through the hierarchical path to find the module:



TestDriver	37.29	56.78	46.64	16.95	15.75	50.33
testHarness	37.29	56.78	46.64	16.95	15.75	50.33
subtree...						



SYNOPSYS

Module Definition

dashboard | **hierarchy** | **modlist** | groups | **tests** | asserts

Module : iteration_div_sqrt_mvp

SCORE	LINE	COND	TOGGLE	FSM	BRANCH
23.33		20.00	0.00		50.00

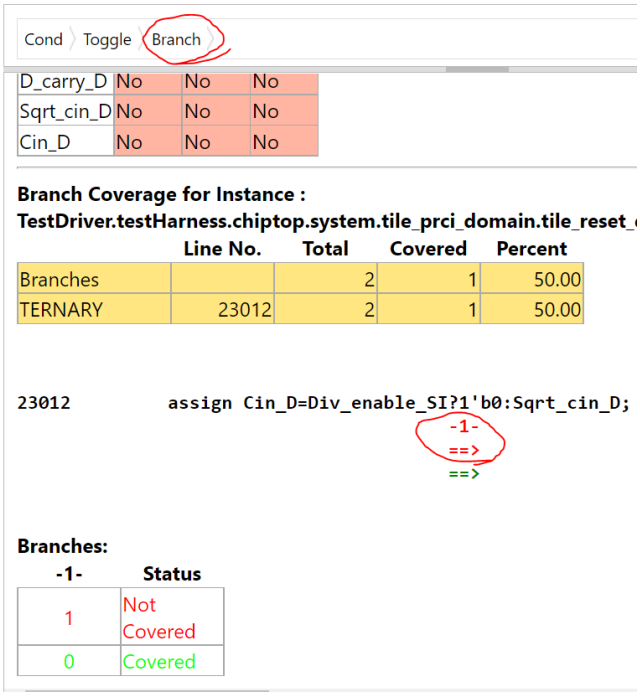
Source File(s) :
/home/grads/c/chenc/test/chipyard/sims/vcs/generated-src/chipyard.TestHarness.CVA6Config/CVA6CoreBlackbox.preproces

Cond | Toggle | Branch

D_carry_D	No	No	No
Sqrt_cin_D	No	No	No
Cin_D	No	No	No

Branch Coverage for Instance :
TestDriver.testHarness.chiptop.system.tile_prci_d

   c. Click *Branch* to find the status of branch points: "-1-" means the id of the branch statements in a module (this is not related to HybridFuzz); "==>" means the uncovered branch point; They are two different things.



Cond | Toggle | Branch

D_carry_D	No	No	No
Sqrt_cin_D	No	No	No
Cin_D	No	No	No

Branch Coverage for Instance :
TestDriver.testHarness.chiptop.system.tile_prci_domain.tile_reset_

	Line No.	Total	Covered	Percent
Branches		2	1	50.00
TERNARY	23012	2	1	50.00

```
23012          assign Cin_D=Div_enable_SI?1'b0:Sqrt_cin_D;
                                              -1-
                                              ==>
                                              ==>
```

Branches:

-1-	Status
1	Not Covered
0	Covered

d. The result shows that there is only one uncovered point in the module. The condition of covering the point is setting *Div_enable_SI* signal to 1.

6. Locate the module in the source code:
   a. Open *CVA6CoreBlackbox.sv*
   b. Search module name: iteration_div_sqrt_mvp

```
assign D_DO[0]=~D_DI[0];
assign D_DO[1]=~(D_DI[1] ^ D_DI[0]);
assign D_carry_D=D_DI[1] | D_DI[0];
assign Sqrt_cin_D=Sqrt_enable_SI&&D_carry_D;
assign Cin_D=Div_enable_SI?1'b0:Sqrt_cin_D;
assign {Carry_out_DO,Sum_DO}=A_DI+B_DI+Cin_D;
```

   c. The result shows that there is only one branch statement in the module.
7. From *cov_html* and source code *CVA6CoreBlackbox.sv,* we can confirm that HybridFuzz.py identifies the correct uncovered point and the conditions of covering it.

The Identification of uncovered points with their conditions

1. Identify number of uncovered points in a module:
   a. Open: cov_xml/snps/coverage/db/testdata/test/branch.verilog.data.xml
   b. Search the module using its hierarchical path.

```
<instance_data name="TestDriver.testHarness.chiptop.system.tile_prci_domain.tile_reset_domain.cva6
.ex_stage_i.fpu_gen.fpu_i.fpu_gen.i_fpnew_bulk.gen_operation_groups[
1].i_opgroup_block.gen_merged_slice.i_multifmt_slice.gen_num_lanes[
0].active_lane.lane_instance.i_fpnew_divsqrt_multi.i_divsqrt_lei.nrbd_nrsc_U0.control_U0.genblk4[
2].iteration_div_sqrt" value="10" />
```

   c. The *value* shows that there are two branch points in the module and one of them is uncovered as shown in “0”.
2. Identify the conditions of covering a point
   a. Open: cov_xml/snps/coverage/db/shape/branch.verilog.shape.xml
   b. Search the module using its module id: branch_def id="-197"

```
<branch_def id="-197" chksum="1770064214" >
  <branch_shape >
    <branch_spec id="0" type="1" totalbranches="2" width="1" file_id="1" skip_annotation="0" chksum="
    1902841359" >
      <branch_expr totalbranches="-2" id="23012" index="1" parent_bit="0" exprstr="Div_enable_SI"
      file_id="1" true_clause_line="23012" false_clause_line="23012" type="0" is_ternary_leaf_node="0" >
      </branch_expr >
      <branch_vector id="0" vector="01" >
      </branch_vector >
      <branch_vector id="1" vector="10" >
      </branch_vector >
    </branch_spec >
  </branch_shape >
</branch_def >
```

   c. This file allows us to obtain the **branch statement tree** of a module and VCS allocates branch points at the leaf nodes only.
   d. The *branch_vector* is corresponding to each branch point. Its vector records the conditions of covering each point. Note: there are some tricky logics to identify conditions from the vectors. We can elaborate in detail during the meeting.
   e. The *branch_expr* is used to record the expression of each branch statement from source codes.
3. Overall, the order of identification: *value* → *branch_vector* → *branch_expr*

Missing functions in the Demo:

Because the Demo is a simplified version, I remove some functions.

1. The functions to exclude unreachable points. Jaspergold can help us identify partial unreachable points during the preprocess stage using its simplification function. The unreachability is usually caused by the configuration of simulation environment. Therefore, when we select points from module, we can prevent from selecting unreachable points.
2. The functions record point selection history. We will record the uncovered points selected when every time switch to formal. There are two reasons to develop this function: (1)we record the number of points selected and their time consumption by Jaspergold to calculate the rate of formal. (2) The test cases from Jaspergold may not cover the corresponding points in simulation. We record those points, so that we will not select them repeatedly. After, we select all uncovered points, we use TheHuzz to fuzz the test cases from Jaspergold for those points. In this case, we will never use Jaspergold for saving time and hope TheHuzz can generate the effective test cases.
3. The functions handle syntax issues of HDL. When load properties to Jaspergold, there will be some syntaxes issues that Jaspergold cannot identify signals in the properties. The function will transfer signals into a recognizable format.

Jaspergold Set Up:

1. Commands:

   $cd JG_setup/

   $source set_up.sh # set up Jaspergold license

   $jg #run Jaspergold; will open a GUI of Jaspergold

   # the following command is in GUI's console

   $include cva6blackbox.tcl # load cva6, initial state, constrained environment, property example.

2. Note: to load properties of other points, check the uncovd_point.json